DELPHI WEB STUDIO PRO

# Runtime Flow

## Following a project from application launch through editing saving and close

Read this as a journey through a running application. Each stage explains what has just happened, which code takes over next and where the flow can branch or stop. The Engineering Guide describes the structure behind these events; this document follows their order.

Windows desktop edition
Last changed November 10, 2026

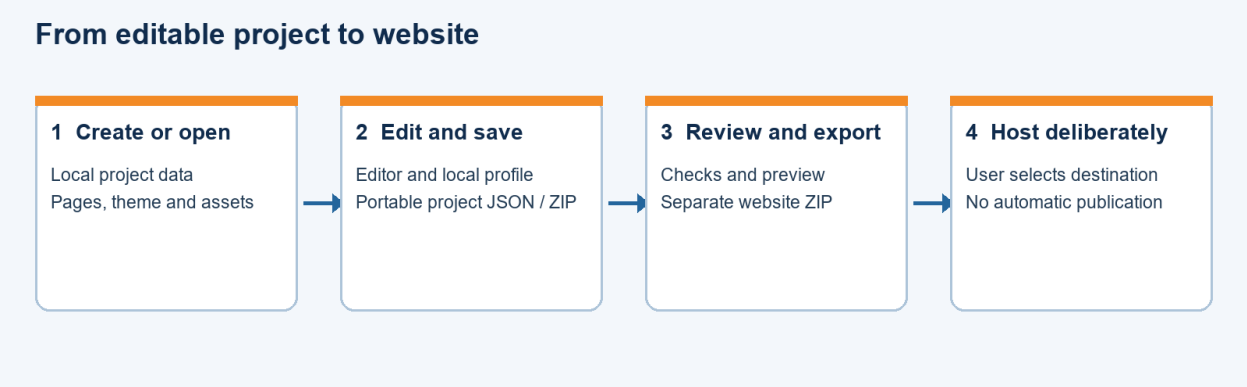
Application source reviewed at revision 61b52c9.

# Contents

The journey through a normal session....3
Windows creates the main form....3
The browser starts React within a local boundary....3
A new or saved project becomes the active draft....4
Editing moves from selection to configuration....5
Media import prepares bytes before placement....6
Preview changes the interaction path....6
Save packages a snapshot and Windows commits it....6
Revisions and recovery take separate routes....7
Export generates the visitor website....7
Optional AI and SFTP branches require their own completion....8
Close waits for the outcome that matters....8

# The journey through a normal session

A session begins in the Windows executable, continues through the embedded editor and reaches a working project only after its configuration is available. From there, an ordinary edit may update a local draft, a save may create a portable file and an export may generate a visitor website. These paths share the project model, but they finish at different places.




*The normal lifecycle branches into local drafting, confirmed saving and independent website output.*

For the examples that follow, imagine opening a practice project, moving a picture, linking some text and saving before close. Optional imports, AI and SFTP branch from that ordinary session when requested. A failure should stop the affected branch with an explanation rather than imply that the next stage has already completed.

# Windows creates the main form

Windows enters DelphiWebStudioPro.dpr. VCL initializes, creates TMainForm from its designer-backed form and begins dispatching events through the application message loop. The DFM supplies the maximized window configuration and connects the controls to their Pascal handlers, so startup appearance and behavior have identifiable native owners.

FormCreate prepares the native session bookkeeping and checks for editor\index.html relative to the executable. When that file is absent, the status explains the missing editor and startup does not pretend to load the application. When it is present, CreateWebView begins an asynchronous WebView2 initialization. Its completion callback verifies the required interfaces and settings before navigating.

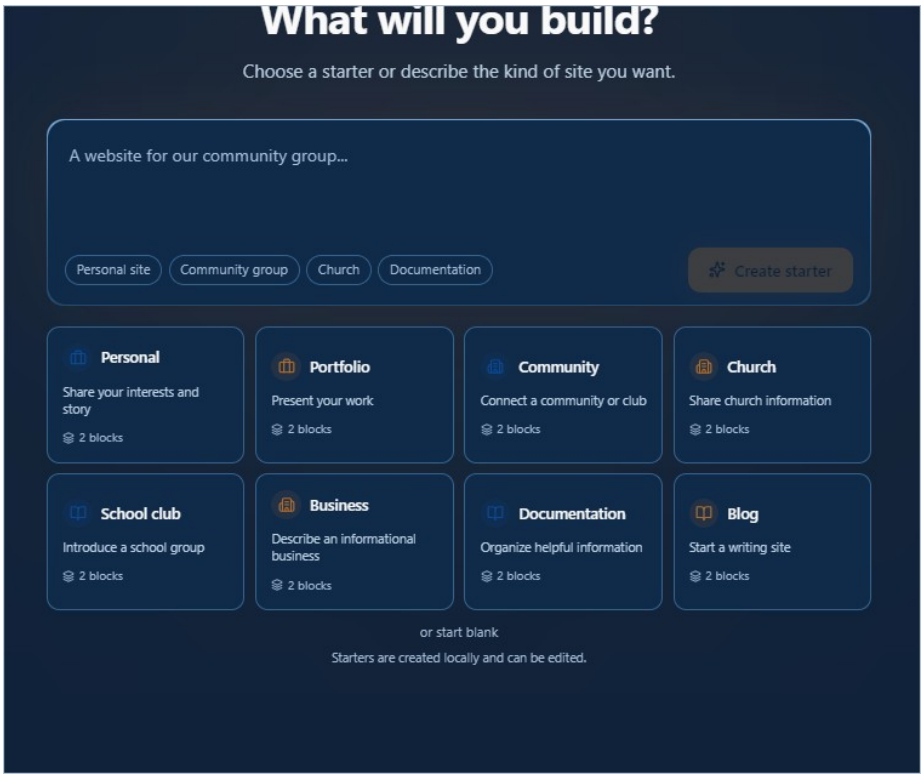
On success, the host maps the adjacent editor folder to https://webstudio.local and loads its index. On failure, the native status remains the useful diagnostic point because the React application may not yet exist. Source path: DelphiWebStudioPro.dpr, WebStudio.Main.dfm, and WebStudio.Main.pas FormCreate and EditorBrowserCreateWebViewCompleted.

# The browser starts React within a local boundary

The local index loads the compiled scripts. src/main.tsx establishes the application root and local-origin policy, then renders App. The desktop origin uses hash routing so the same local document can display

Dashboard, Editor, Export and Settings without asking the native host to serve a different disk file for each route.

The stores read available persisted data and establish the session's UI state. Guarded draft storage preserves detected unreadable values rather than silently replacing them with a new empty project list. Media bytes and local revisions are opened through their own IndexedDB modules as needed. The Dashboard can now show available drafts and ways to begin.




***Once the editor has initialized, the Dashboard presents local drafts and project entry commands.***

Native navigation events continue to govern the boundary. Navigation revokes an existing bridge session, and destination, frame, new-window and permission checks determine what can proceed. The privilege available to the local editor is therefore checked again at the native message boundary rather than assumed from the appearance of a page. Source path: src/main.tsx, App.tsx, the three store modules and WebStudio.Main.pas navigation handlers.

## A new or saved project becomes the active draft

Choosing a starter creates a local configuration and draft identity, selects it and opens Editor. The description-based starter path chooses local starter content; a cloud AI request does not occur merely because a description was typed. A project card later returns to that stored draft, while Open project starts from an independently chosen portable file.

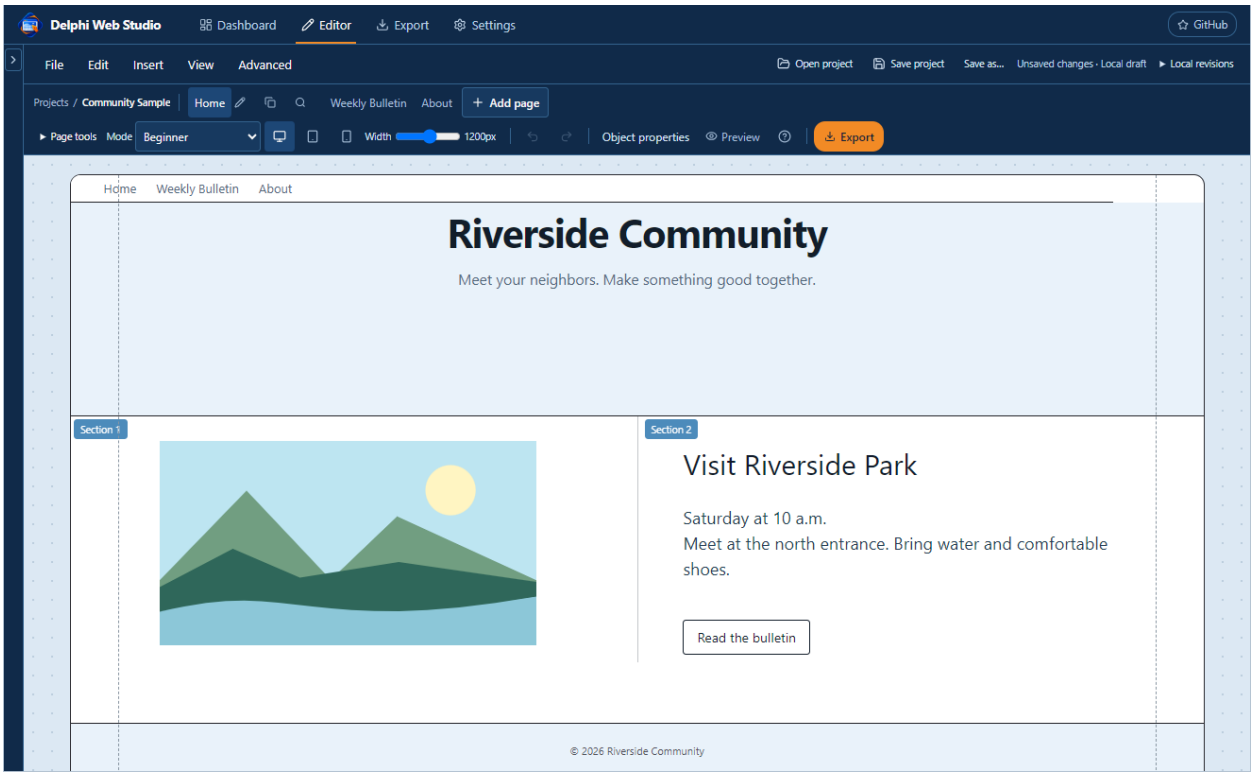
For a portable file, ProjectFileCommands requests the appropriate desktop capability, parses supported JSON or the ZIP manifest, and validates the data. ZIP checks include the manifest bounds and CRC before strict UTF-8 decoding. Associated media is checked and restored before the candidate replaces the active configuration.

The open path first compares the saved snapshot with drafts already known to the app. A matching saved version reuses its local draft, preserving newer local edits. A different version can open separately. If the current session changes during preparation or a media check fails, the candidate must not silently replace that working session.

HTML and WordPress imports branch through their own previews and reports before creating a separate converted project. Their source files remain reference material. Source path: Dashboard.tsx, ProjectFileCommands.tsx, project-file.ts, static-html-import.ts, wordpress-import.ts and WebStudio.ProjectOpen.pas.

## Editing moves from selection to configuration

Editor chooses the active page and Canvas renders its registered blocks. A page tab or Go to page changes activePageId; the fixed command group then resolves that identity when Rename, Duplicate or another page action runs. The surrounding editor stores selection and view state separately from the saved content.




*The active configuration is rendered as a page whose objects can be selected independently.*

A single click identifies the block or object, and its outline connects to movement, handles and focused settings. A double-click on supported text enters editing. If objects overlap, Layers and alternate selection identify the intended item without changing its content. Store actions commit geometry or text changes and record the appropriate undo snapshot.

Panel ownership is carried by parentId, so a panel move also moves its attached contents. A collage has its own image positions and layers. Section divider movement updates the saved proportions, and

responsive rendering can later turn that desktop arrangement into rows or a vertical stack. Source path: Editor.tsx, Canvas.tsx, BlockWrapper.tsx, BlockColumns.tsx, SectionObjects.tsx and configStore.ts.

## Media import prepares bytes before placement

Opening the media picker does not create a second media collection. It reads the same project-scoped assets used by Manage Media. Import validates the selected files, applies the duplicate policy and records supported bytes and metadata, including sizes and hashes. Cancellation can stop the remaining batch while leaving files already imported available.

Folder moves update organization without changing asset IDs or portable paths. Choosing a file for an existing property returns its reference to that property; Add to page creates the supported object or arms image placement. Canvas resolves the persistent reference into a temporary object URL for display and releases those session URLs when they are no longer needed.

A later save packages the original bytes, not the temporary display URL. That is why moving a photograph between logical folders does not require reconnecting every page that uses it. Source path: MediaLibraryDialog.tsx, MediaPanel.tsx, media-library.ts and Canvas.tsx.

## Preview changes the interaction path

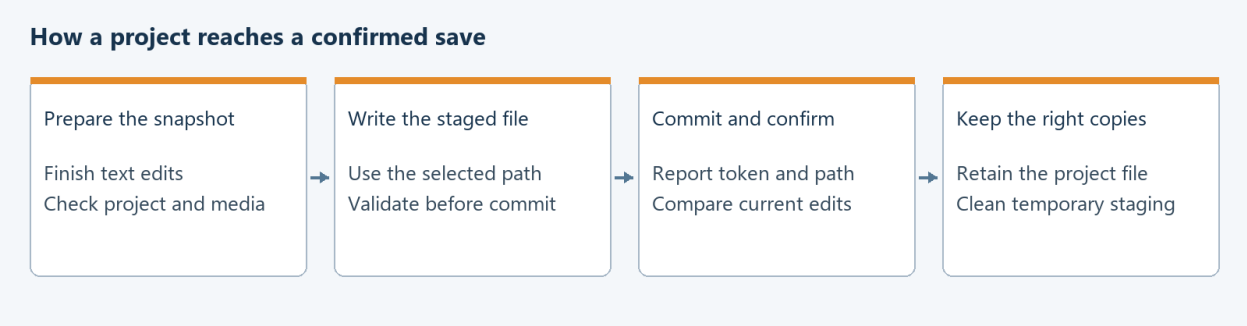
When Preview is enabled, the configured content remains but authoring behavior is disabled. A mouse click should now activate a visitor link instead of selecting a text box. Editing-only pointer capture returns early, preventing a selection update from replacing the clicked anchor between pointer-down and pointer-up.

An internal link resolves to an eligible page and any optional anchor. For an ordinary page change, Canvas resets its scroll container to the top in a layout effect. For a named destination, it resolves the pending anchor instead. A same-page plain link also resets the position, while an anchor can intentionally scroll to a lower point.

Navbar dropdowns, calendar months, enlarged gallery photos and biography readers maintain visitor UI state without authoring a new project version. Escape on the Navbar closes the open dropdown and returns focus to its arrow. Real mouse-event tests are valuable here because a scripted click skips part of the event sequence. Source path: Canvas.tsx and the Navbar, Calendar, Gallery and Team renderers and shared scripts.

## Save packages a snapshot and Windows commits it

Save begins by finishing inline text edits and capturing a consistent snapshot of the configuration and settings. The browser requests project.save authorization, validates the project and required media, then creates JSON or a project ZIP. The ZIP contains project.webstudio.json and original assets. A token identifies the snapshot as it passes to the native download path.




***The portable save travels through packaging, native staging and confirmation, while revisions and backups remain separate branches.***

The desktop host uses the known project path for Save or opens the destination chooser for Save as. It stages bytes beside that destination, validates the staged project and checks the existing file before committing an update. Successful cleanup removes temporary replacement material. An interrupted or failed operation may leave an identified stage that requires inspection.

After commit, project.save.complete returns the path and token. The editor compares the completed snapshot with current state and input version, then reports whether the project is clean. A new edit during saving stays dirty even when the earlier snapshot reached disk successfully. Source path: ProjectFileCommands.tsx, native-bridge.ts, WebStudio.Main.pas and WebStudio.ProjectSave.pas.

## Revisions and recovery take separate routes

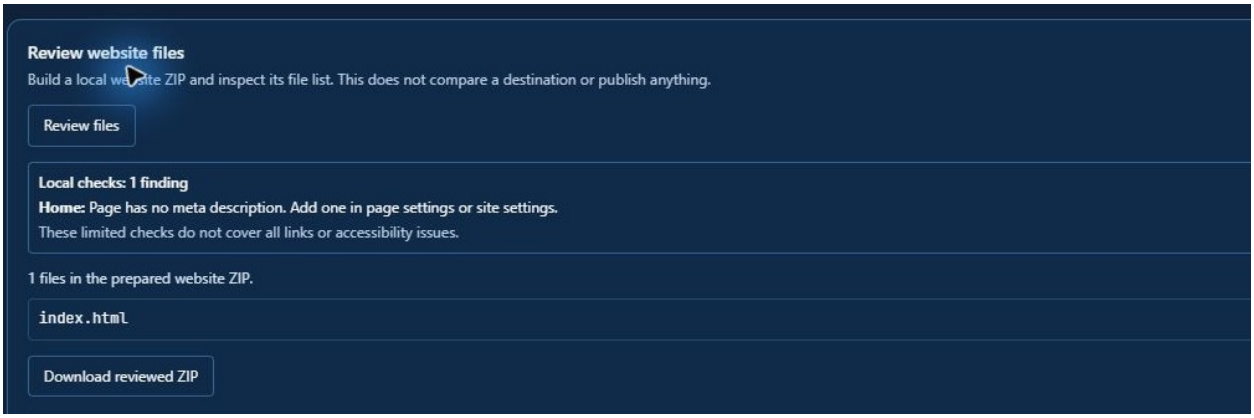
A local revision stores a snapshot in its own IndexedDB collection, where it can be listed or saved as a portable file. It is useful within the same profile but does not become an independent disk backup just because the revision command completed. A failed revision write should preserve the earlier completed entries.

The native backup command instead packages an already selected disk project, and restore writes a separate destination. Recover staged file works only when suitable retained material exists; recognizing its initial bytes as JSON or ZIP does not establish that the full archive is complete. Reopen a recovered project and inspect its pages and media before relying on it.

Ordinary successful saves no longer create an accumulating recovery folder for every update. When cleanup cannot be completed or a replacement needs investigation, the reported path identifies what remains. Source path: project-revisions.ts, WebStudio.Main.pas backup and recovery handlers, and the project save units.

## Export generates the visitor website

The Export route captures the active configuration for review and generation. Supported blocks become page HTML, links are rewritten for their generated file depth and referenced assets are included. Draft articles remain in the editable project but are omitted from visitor output. A project change after review requires another review of the current version.




*Website review generates a local set of files. Downloading or reviewing them does not transfer them to a public host.*

The website ZIP and project ZIP finish different jobs. The former is intended to be served to visitors and does not contain the full editing environment. The latter retains editable structure, drafts and media. The Windows review can serve exported files locally over HTTP so PDF modules and other browser features can be tested under normal website conditions.

Neither export nor the GitHub software link creates a repository or enables public hosting. An SFTP operation follows a separate reviewed path. Source path: Deploy.tsx, export-html.ts, export.css, website-manifest.ts and the desktop website-review and deployment units.

## Optional AI and SFTP branches require their own completion

Local AI checks the selected project and runtime inputs before starting background work. The native host allocates a request identity, marks the operation busy and invokes the local planning process. Its worker returns through the UI thread, clears the working state and presents either a result or a failure. A result is still a proposal until the editor validates and the user accepts it.

Cloud generation uses a separate network path. A Gemini key can be retrieved from Windows Credential Manager for the current account, while session consent controls whether cloud generation is enabled. A successful key test confirms that particular response, not every model, quota or billing condition.

SFTP connection test, destination scan, backup, review, preflight, execution and rollback are distinct steps. A transfer requires the reviewed inputs and destination to remain valid. If it fails partway, the receipt and recovery material help determine what changed before another operation is attempted. Source path: WebStudio.LocalAIHost.pas, LocalPlanReview.tsx, site-actions.ts, Settings.tsx and the desktop deployment units.

## Close waits for the outcome that matters

File > Exit and the window close button enter the same native close guard. Active local AI work and save activity have their own rules. For unsaved changes, Yes requests a save and waits for the completed

clean confirmation; No allows leaving without another file update, and Cancel keeps the session open. A failed or cancelled save must not be treated as permission to discard the work.

When the form is destroyed, the native bridge grant is revoked and its session is freed. Test profiles, local review servers and owned child processes have separate cleanup responsibilities in their respective paths. A normal successful close is therefore the end of a sequence, not proof that every earlier optional operation succeeded.

This flow describes the inspected implementation. Controlled tests exercise particular timings and boundaries, while clean Windows installation, unusual devices and actual publication require separate acceptance. Read a completion result in terms of the operation it confirms, and preserve the editable master when moving on to another stage of work.